Database Fundamentals

Examples, Aggregate Functions and Grouping

Dr. Sergio Pisano

This presentation contains adapted materials from different sources, including our designated textbook, to be used for educational purposes only

Summary

- Detailed Query Examples
- Aggregate Functions and Grouping
- α (Alpha) Aggregation Operator
- α (Alpha) Operator: Two Arguments

Relational Operators Review:

- Selection **σ** (sigma)
- Projection **π** (pi)
- Union by **U**
- Intersection **∩**
- Set-difference **–**
- Cartesian (cross) product **x**
- Renaming **ρ** (rho)
- JOINs ⋈
- DIVISION **/**

Schema Example: Example of auction of cars by drivers

DRIVERS

d_id	dname	rating	age

BIDS

d_id	c_id	date	price

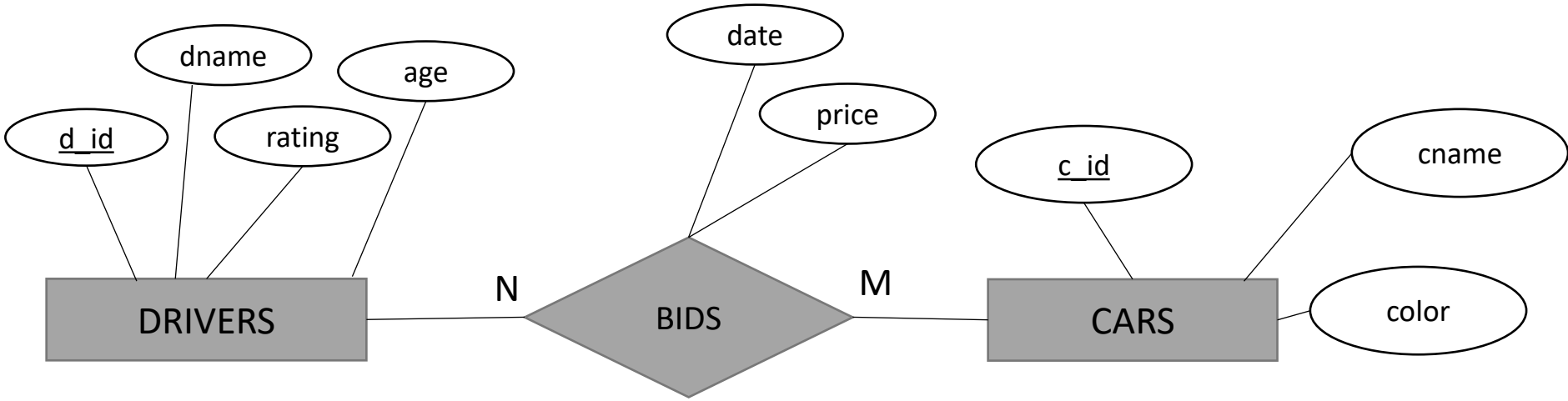
CARS

c_id	cname	color

Drivers (d_id: int, dname: varchar, rating: int, age: numeric);

Cars (c_id:int, cname: varchar, color: varchar);

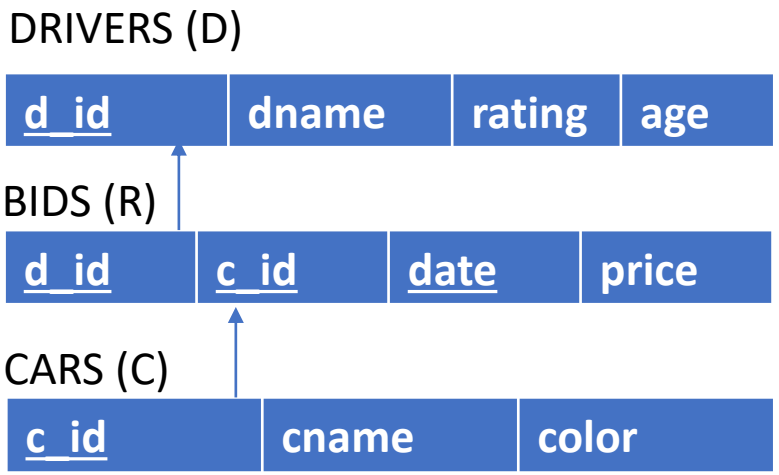
Bids (d_id: int, c_id: int, day: date, price: numeric);



A CAR can receive bids from many DRIVERS

A DRIVER can bid on many CARS

Q1: Find the colors of cars with bids by Dustin



The query involves all 3 related tables:

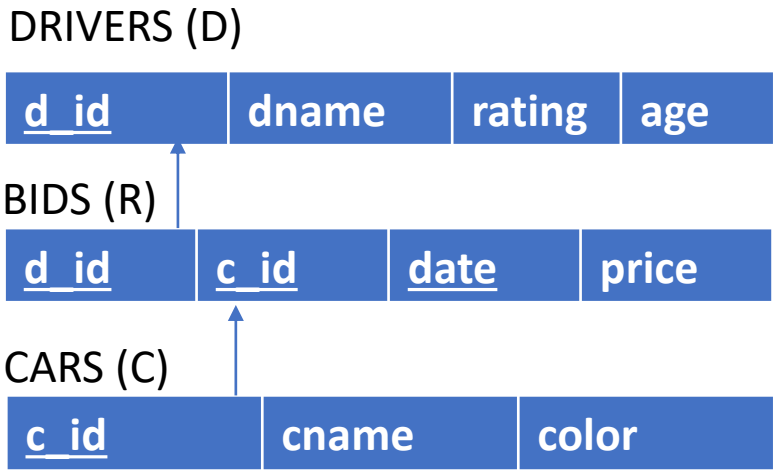
- Color is contained in **CARS**
- Driver name is contained in **DRIVERS** (condition is here)
- Drivers' bids are contained in **BIDS**

STEP1: DRIVERS1 = $\sigma_{\text{dname='Dustin'}}$ (DRIVERS)

STEP2: π_{color} (DRIVERS1 * BIDS * CARS)

NATURAL JOINS

Q2: Find all drivers ids of drivers who have a rating of at least 8 OR bid at car 103.



The query involves all 3 related tables:

- Drivers id's are in **DRIVERS**
- Ratings are in **DRIVERS** (condition)
- Bids for cars are in **BIDS** (condition)

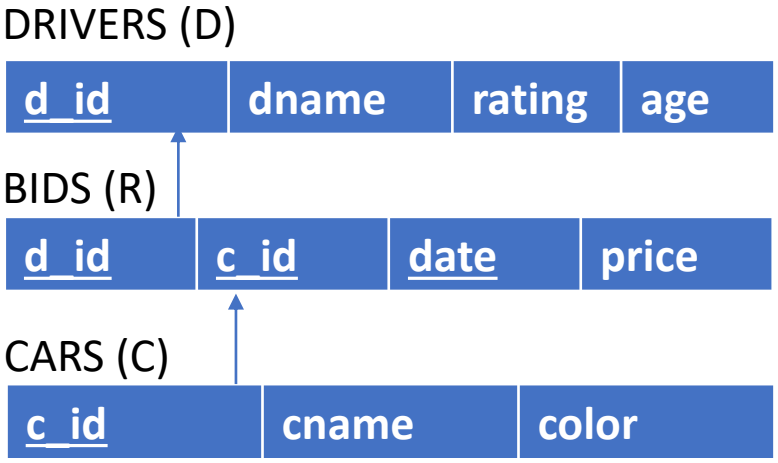
STEP1: DRIVERS1 = $\sigma_{rating=>8}$ (DRIVERS) ← all drivers who have a rating of at least 8

STEP2: BIDS1 = $\sigma_{c_id=103}$ (BIDS) ← all drivers who have bid at car 103

Next we need to combine both DRIVERS1 and BIDS1 (DRIVERS1 and BIDS1 must be made union compatible)

STEP3: RESULT = π_{d_id} (DRIVERS1) U π_{d_id} (BIDS1)

Q3: Find the names of drivers who have bids on a red car



The query involves all 3 related tables:

- Drivers names are in **DRIVERS**
- Car color is in **CARS** (condition)
- Bids for cars are in **BIDS**

STEP1: REDCARS = $\sigma_{color='Red'}(CARS)$

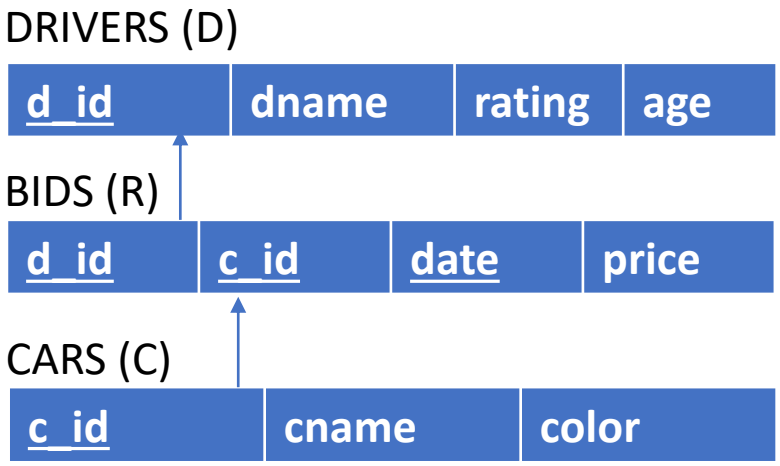
Next we need to connect (join) REDCARS with BIDS (R)

STEP2: REDBIDS = REDCARS $\bowtie$ BIDS

Next I need to connect REDBIDS with DRIVERS

STEP3: RESULT = $\pi_{dname}(\pi_{d_id}(REDBIDS) \bowtie DRIVERS)$

Q4: Find the names of drivers who have bids on any car



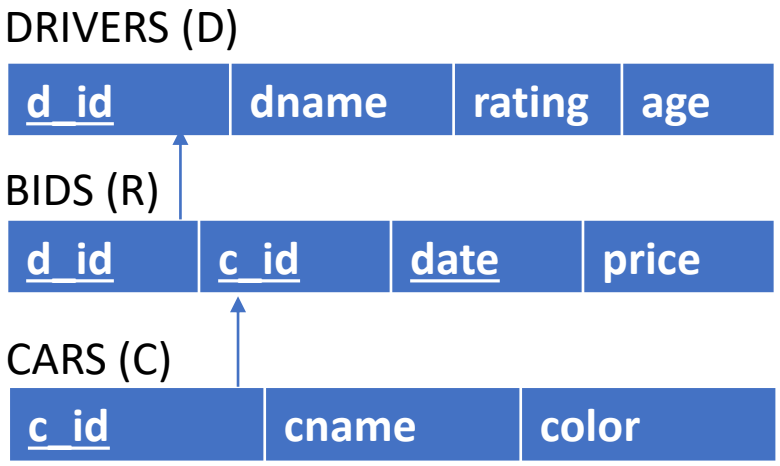
The query involves 2 related tables:

- Drivers names are in **DRIVERS**
- Bids for cars are in **BIDS**

We need to connect DRIVERS with BIDS:

RESULT = πdnames(DRIVERS $\bowtie$ BIDS)

Q5: Find the names of drivers who have NO bids on a red car



The query involves all 3 related tables:

- Drivers names are in **DRIVERS**
- Car color is in **CARS** (condition)
- Bids for cars are in **BIDS**

$$\pi_{dname}\left(\left(\pi_{d_id}(DRIVERS) - \pi_{d_id}\left(\sigma_{color='Red'}(CARS) \bowtie BIDS\right)\right) \bowtie DRIVERS\right)$$

1. Find all red cars
2. Find all bids on red cars
3. Subtract from all drivers, the ones with bids on red cars to get the ones with bids on cars with other colors (make sets union compatible first by projecting d_id)
4. Join result with drivers to get access driver names
5. Project drivers' names

Q6: Find the id's of drivers with age over 20 who have NOT bid on a red car

DRIVERS (D)

d_id	dname	rating	age

BIDS (R)

d_id	c_id	date	price

CARS (C)

c_id	cname	color

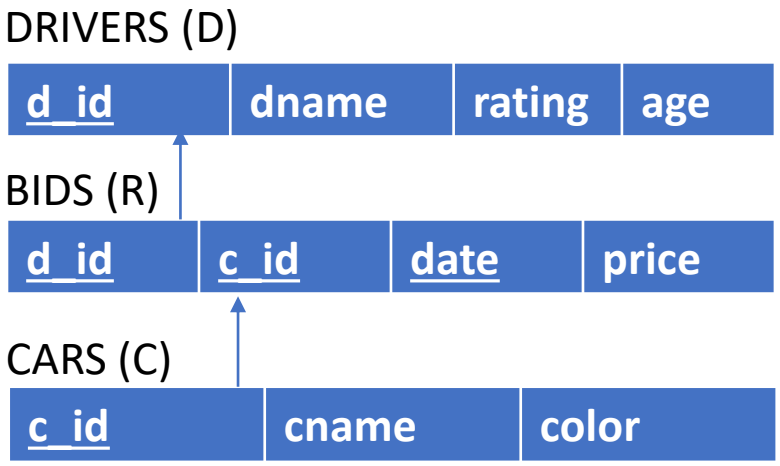
The query involves all 3 related tables:

- Drivers ID's and age is in **DRIVERS** (condition)
- Car color is in **CARS** (condition)
- Bids for cars are in **BIDS**

$$\pi_{d_id}(\sigma_{age>20}(DRIVERS)) - \pi_{d_id}(\sigma_{color='Red'}(CARS) \bowtie BIDS)$$

- Find the red CARS that have bids (join) first
- Find the DRIVERS over 20 and subtract the red CARS with bids from the set
- Because subtraction needs to be union compatible, we must project the d_id from both sets before subtracting

Q7A: Find the names of drivers who have bid on a red OR a blue car



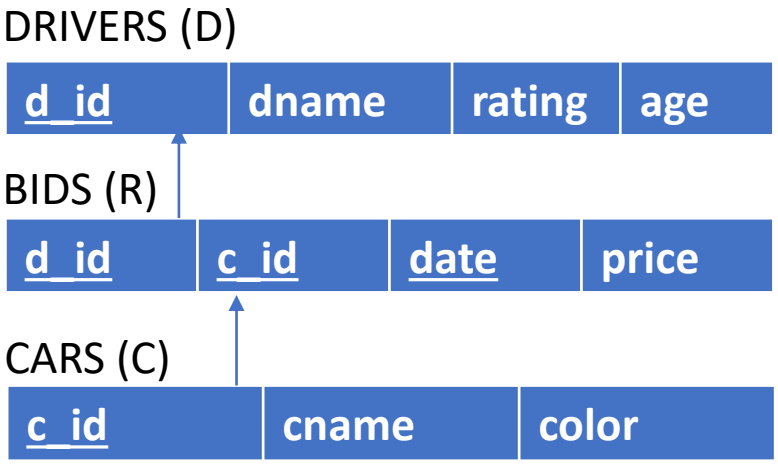
The query involves all 3 related tables:

- Drivers names are in **DRIVERS**
- Bids for cars are in **BIDS**
- Car colors are in **CARS** (condition)

$$\pi_{\text{dname}}(\sigma_{\substack{\text{color='Red'} \\ \vee\ \text{color='Blue'}}}(\text{CARS}) \bowtie \text{BIDS} \bowtie \text{DRIVERS})$$

- Find the red or blue CARS that have bids (join) first
- Find the DRIVERS related to those bids (join)

Q7B: Find the names of drivers who have bid on a red OR a blue car



Instead of using an additional Boolean condition in the SELECT, we can also rewrite the Query using an **UNION**

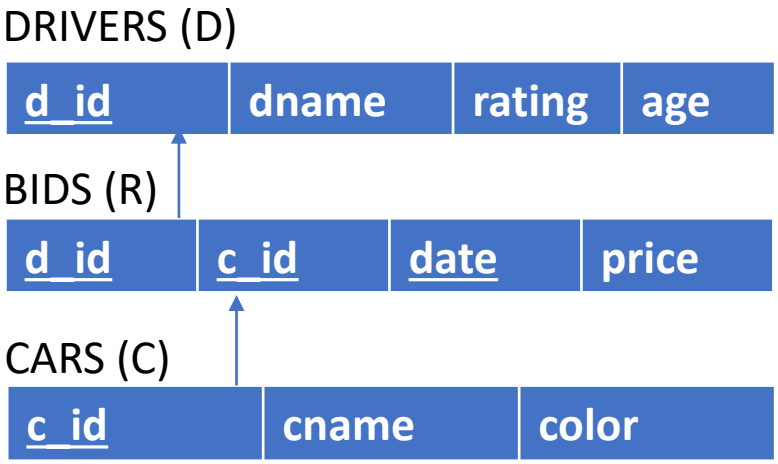
$$\textbf{REDBLUE} = (\sigma_{color='Red'}(CARS) \cup \sigma_{color='Blue'}(CARS))$$

We can re-express this as:

$$\rho\ (\textbf{REDBLUE}, (\sigma_{color='Red'}(CARS) \cup \sigma_{color='Blue'}(CARS)))$$

$$\textbf{RESULT} = \pi_{dname}(REDBLUE \bowtie BIDS \bowtie DRIVERS)$$

Q8: Find the names of drivers who have bid on a red AND blue car



Since we want the results that coincide when overlapping (joining) 2 compatible sets, we need to use Intersection

$$\textbf{TEMP1} = \pi_{\text{d_id}}(\sigma_{\text{color='Red'}}(\text{CARS}) \bowtie \text{BIDS})$$

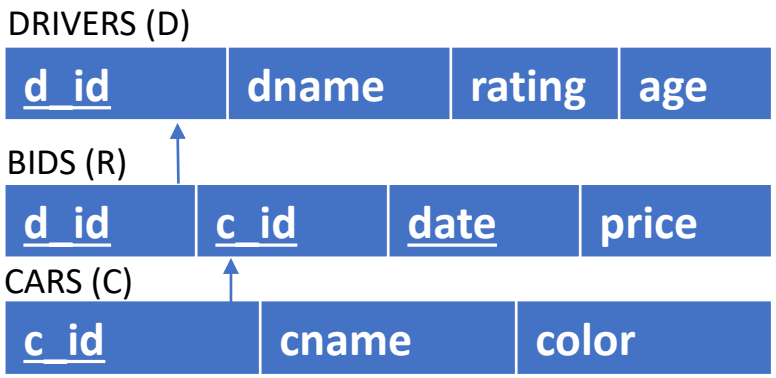
$$\textbf{TEMP2} = \pi_{\text{d_id}}(\sigma_{\text{color='Blue'}}(\text{CARS}) \bowtie \text{BIDS})$$

$$\textbf{TEMP3} = (\text{TEMP1} \cap \text{TEMP2})$$

$$\textbf{RESULT} = \pi_{\text{dname}}(\text{TEMP3} \bowtie \text{DRIVERS})$$

1. We get all red cars with bids
2. We get all blue cars with bids
3. We intersect the two sets and get the bids that have blue **AND** red cars
4. Finally, to get the names of the drivers we join TEMP3 with DRIVERS and project the name

Q9: Find the names of drivers who have bids on at least 2 cars



The query involves all 2 related tables:

- Drivers names are in **DRIVERS**
- Bids for cars are in **BIDS**

Because the query is asking us to **compare data from BIDS with data from BIDS** (**itself**) twice, we need to use a SELF join

BIDS

d_id	c_id	date	price
22	101	2018-10-10	$500
58	103	2018-11-12	$900
22	102	2017-5-13	$700

⋈ d_id

BIDS

d_id	c_id	date	price
22	101	2018-10-10	$500
58	103	2018-11-12	$900
22	102	2017-5-13	$700

=

JOIN STEP1: BIDS X BIDS

d_id	c_id	date	price	d_id	c_id	date	price
22	101	2018-10-10	$500	22	101	2018-10-10	$500
22	101	2018-10-10	$500	58	103	2018-11-12	$900
22	101	2018-10-10	$500	22	102	2017-5-13	$700
58	103	2018-11-12	$900	22	101	2018-10-10	$500
58	103	2018-11-12	$900	58	103	2018-11-12	$900
58	103	2018-11-12	$900	22	102	2017-5-13	$700
22	102	2017-5-13	$700	22	101	2018-10-10	$500
22	102	2017-5-13	$700	58	103	2018-11-12	$900
22	102	2017-5-13	$700	22	102	2017-5-13	$700

JOIN STEP2: $\sigma_{d_id=d_id}$ (BIDS x BIDS)

d_id	c_id	date	price	c_id	date	price
22	101	2018-10-10	$500	101	2018-10-10	$500
22	101	2018-10-10	$500	102	2017-5-13	$700
58	103	2018-11-12	$900	103	2018-11-12	$900
22	102	2017-5-13	$700	101	2018-10-10	$500
22	102	2017-5-13	$700	102	2017-5-13	$700

- Notice that only one d_id field remained after the JOIN in STEP2
- The field name c_id that we are interested in our query repeats in the results (STEP2)
- To express this in relational algebra, we need to apply renaming to that field first

JOIN STEP2: $\sigma_{d_id=d_id}$ (BIDS x BIDS)

d_id	cid1	date	price	cid2	date	price
22	101	2018-10-10	$500	101	2018-10-10	$500
22	101	2018-10-10	$500	102	2017-5-13	$700
58	103	2018-11-12	$900	103	2018-11-12	$900
22	102	2017-5-13	$700	101	2018-10-10	$500
22	102	2017-5-13	$700	102	2017-5-13	$700

- We rename c_id in position 2 to cid1 and c_id in position 5 to cid2
- We then select the tuples where cid1 is different from cid2

$$\rho\ (\text{TEMP}\ (2\text{-> cid1, 5->cid2}))\ ,\ \text{BIDS} \bowtie_{d_id} \text{BIDS})$$

$$\pi_{\text{dname}}\ (\pi_{d_id}\ (\sigma_{\text{cid1}\neq\text{cid2}}\ (\text{TEMP}))) \bowtie \text{DRIVERS})$$

FINAL RESULT:

dname
Dustin

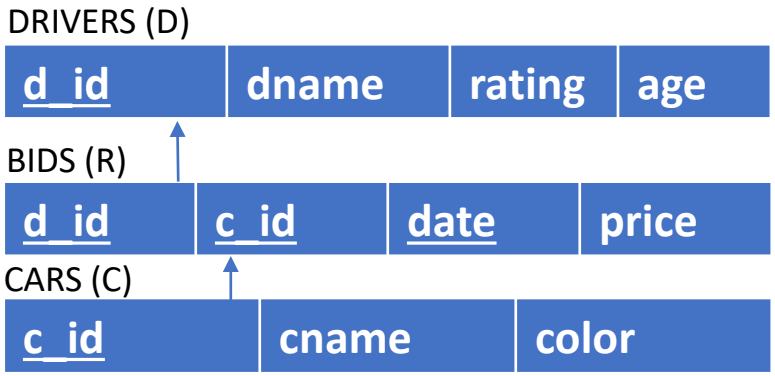
$\sigma_{\text{cid1}\neq\text{cid2}}$ (TEMP))

d_id	cid1	date	price	cid2	date	price
22	101	2018-10-10	$500	102	2017-5-13	$700
22	102	2017-5-13	$700	101	2018-10-10	$500

$\pi_{d_id}\ (\sigma_{\text{cid1}\neq\text{cid2}}\ (\text{TEMP}))$

d_id
22

Q10: Find the names of drivers who have bid ON ALL cars.



Because we want to find ALL instances of records from one table (CARS) in another (BIDS), we must use **DIVISION**

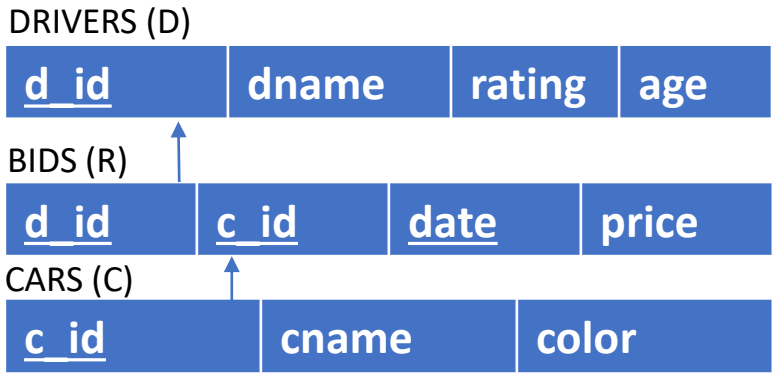


Note: we removed the fields we do not want to keep

$$\textbf{TEMP1} = \pi_{d_id,\ c_id}(\text{BIDS}) \ / \ \pi_{c_id}(\text{CARS})$$

$$\textbf{RESULT} = \pi_{dname}(\text{TEMP1} \bowtie \text{DRIVER})$$

Q11: Find the names of drivers who have bid on all cars called 'Mustang'



The query involves all 3 related tables:

- Drivers names are in **DRIVERS**
- Bids for cars are in **BIDS**
- Car names are in **CARS** (condition)

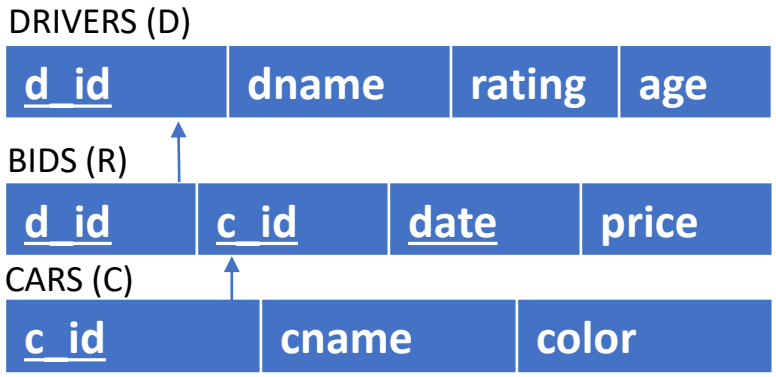
$$\textbf{TEMP1} = \left(\pi_{\text{c_id}} \left(\sigma_{\text{cname='Mustang'}} (\text{CARS})\right) \bowtie \text{BIDS}\right)$$

$$\textbf{TEMP2} = \left(\pi_{\text{c_id,d_id}} (\text{BIDS}) \,/\, \pi_{\text{c_id}} \text{TEMP1}\right)$$

$$\textbf{RESULT} = \left(\pi_{\text{dname}} (\text{DRIVERS} \bowtie \text{TEMP2})\right)$$

1. First, we get the card IDs for all cars named Mustang
2. Join that with BIDS to get all Mustang's with bids and PROJECT c_id
3. Divide all BIDS (PROJECT d_id,c_id) by All Mustang's with bids to get all d_id's of drivers that have bid on all Mustangs (TEMP2)
4. Finally, join TEMP2 with DRIVERS and project their names

Q12: Find the driver id's of drivers whose rating is better than ANY driver named Jack



The query involves only 1 table:

- Drivers names & ratings are in **DRIVERS (condition)**
- Notice I need to compare **DRIVERS** with **itself**

$$\textbf{TEMP1} = (\rho_{d2}(\text{DRIVERS}) \times \sigma_{\text{dname='Jack'}}(\text{DRIVERS}))$$

$$\textbf{RESULT} = \pi_{d2.d_id}(\sigma_{\text{d2.rating>DRIVERS.rating}}(\text{TEMP1}))$$

1. First, we select the record(s) for drivers named 'Jack'
2. Next, we need all combinations of DRIVERS with drivers named 'Jack' so we can compare their ratings. We also need extra fields in our result (for Jack's records) since we can not compare the drivers' rating field with itself
3. Therefore, we must use the CARTESIAN PRODUCT and rename the duplicate field(s)
4. Then we can compare the ratings of d2 with DRIVERS to get the solution

(Better than ANY driver named Jack)

Instance of DRIVERS

d_id	dname	rating	age
22	Dustin	25	45
67	Jack	10	35
56	Jack	10	28
44	Jack	20	21
32	Pedro	15	19

1. $\sigma_{\text{dname='Jack'}}(\text{DRIVERS})$

d_id	dname	rating	age
67	Jack	10	35
56	Jack	10	28
44	Jack	20	21

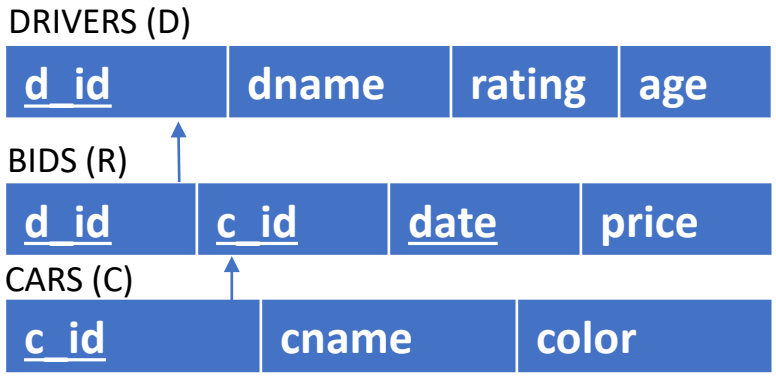
d2.d_id
22
44
32

2. TEMP1 = $(\rho_{\text{d2}}(\text{DRIVERS}) \times \sigma_{\text{dname='Jack'}}(\text{DRIVERS}))$

d2.d_id	d2.dname	d2.rating	d2.age	d_id	dname	rating	age
22	Dustin	25	45	67	Jack	10	35
22	Dustin	25	45	56	Jack	10	28
22	Dustin	25	45	44	Jack	20	21
67	Jack	10	35	67	Jack	10	35
67	Jack	10	35	56	Jack	10	28
67	Jack	10	35	44	Jack	20	21
56	Jack	10	28	67	Jack	10	35
56	Jack	10	28	56	Jack	10	28
56	Jack	10	28	44	Jack	20	21
44	Jack	20	21	67	Jack	10	35
44	Jack	20	21	56	Jack	10	28
44	Jack	20	21	44	Jack	20	21
32	Pedro	15	19	67	Jack	10	35
32	Pedro	15	19	56	Jack	10	28
32	Pedro	15	19	44	Jack	20	21

3. **RESULT** = $\pi_{\text{d2.d_id}}(\sigma_{\text{d2.rating>DRIVERS.rating}}(\text{TEMP1}))$

Q13: Find the driver id's of drivers whose rating is better than ALL drivers named Jack



The query involves only 1 table:

- Drivers names & ratings are in **DRIVERS (condition)**
- Notice I need to compare **DRIVERS** with **itself**

$$\textbf{TEMP1} = (\rho_{d2}(\text{DRIVERS}) \times \sigma_{\text{dname='Jack'}}(\text{DRIVERS}))$$

$$\textbf{TEMP2} = \sigma_{\text{d2.rating} \leqslant \text{DRIVERS.rating}}(\text{TEMP1})$$

$$\textbf{RESULT} = \pi_{\text{d_id}}(\text{DRIVERS}) - \pi_{\text{d2.d_id}}(\text{TEMP2})$$

1. First, we select the record(s) for drivers named 'Jack'
2. Next, we need all combinations of DRIVERS with drivers named 'Jack' so we can compare their ratings. We also need extra fields in our result (for Jack's records) since we can not compare the drivers' rating field with itself
3. Therefore, we must use the CARTESIAN PRODUCT and rename the duplicate field(s)
4. Then we can compare the ratings and use the set difference with DRIVERS

(Better than ALL drivers named Jack)

Instance of DRIVERS

d_id	dname	rating	age
22	Dustin	25	45
67	Jack	10	35
56	Jack	10	28
44	Jack	20	21
32	Pedro	15	19

1. $\sigma_{\text{dname='Jack'}}$ (DRIVERS)

d_id	dname	rating	age
67	Jack	10	35
56	Jack	10	28
44	Jack	20	21

3. TEMP2 = $\sigma_{\text{d2.rating} \leqslant \text{DRIVERS.rating}}$ (TEMP1)

2. TEMP1 = (ρ_{d2} (DRIVERS) X $\sigma_{\text{dname='Jack'}}$ (DRIVERS))

d2.d_id	d2.dname	d2.rating	d2.age	d_id	dname	rating	age
22	Dustin	25	45	67	Jack	10	35
22	Dustin	25	45	56	Jack	10	28
22	Dustin	25	45	44	Jack	20	21
67	Jack	10	35	67	Jack	10	35
67	Jack	10	35	56	Jack	10	28
67	Jack	10	35	44	Jack	20	21
56	Jack	10	28	67	Jack	10	35
56	Jack	10	28	56	Jack	10	28
56	Jack	10	28	44	Jack	20	21
44	Jack	20	21	67	Jack	10	35
44	Jack	20	21	56	Jack	10	28
44	Jack	20	21	44	Jack	20	21
32	Pedro	15	19	67	Jack	10	35
32	Pedro	15	19	56	Jack	10	28
32	Pedro	15	19	44	Jack	20	21

(Better than ALL drivers named Jack)

3. TEMP2 = $\sigma_{d2.rating \leqslant DRIVERS.rating}$ (TEMP1)

d2.d_id	d2.dname	d2.rating	s2.age	d_id	dname	rating	age
67	Jack	10	35	67	Jack	10	35
67	Jack	10	35	56	Jack	10	28
67	Jack	10	35	44	Jack	20	21
56	Jack	10	28	67	Jack	10	35
56	Jack	10	28	56	Jack	10	28
56	Jack	10	28	44	Jack	20	21
44	Jack	20	21	44	Jack	20	21
32	Pedro	15	19	44	Jack	20	21

4-. Instance of DRIVER

d_id	dname	rating	age
22	Dustin	**25**	45
67	Jack	10	35
56	Jack	10	28
44	Jack	20	21
32	Pedro	15	19

Checking in this example we can see how Dustin is the driver whose rating is higher than ALL Jacks

π_{d_id} (DRIVERS)

d_id
22
67
56
44
32

−

$\pi_{d2.d_id}$ (TEMP2)

d2.d_id
67
56
44
32

=

4-. RESULT

d_id
22

RESULT = π_{d_id} (DRIVERS) - $\pi_{d2.d_id}$ (TEMP2)

Aggregate Functions and Grouping

- So far we have focused on retrieving data
- Calculations (e.g. counting, averaging, etc.) are often needed
- Examples
  - Finding the average age of all drivers
  - Find the number of drivers that have reserved a red car
- **Aggregate functions** can be used in queries (e.g. **COUNT, AVG, MIN, MAX**)
- To get the results we want, we must often organize results in “groups”

α (Alpha) Aggregation Operator

- Allow us to apply an aggregate function to a set
- Syntax:

$$\alpha_{\text{Arg1}}(R)$$

- Arg1 represents aggregate function applied to a specific fields in R

α (Alpha) Operator

Q14: find the average age of all drivers

$$\alpha_{AVG(age)}(DRIVERS)$$

Instance of DRIVERS

d_id	dname	rating	age
22	Dustin	25	45
67	Jack	10	35
56	Jack	10	28
44	Jack	20	21
32	Pedro	15	19

RESULT

age
29.6

α (Alpha) Operator

Q15: Find the number of drivers that bid on a red car

$$\alpha_{COUNT(d_id)} \left(\sigma_{color='Red'} (BIDS \bowtie CARS) \right)$$

Instance of CARS

c_id	cname	Color
101	Mustang	Red
102	Mustang	Blue
103	Malibu	Green
104	Focus	Red

Instance of BIDS

d_id	c_id	date
22	101	2018-10-10
58	104	2017-5-11
33	101	2017-1-5
87	102	2017-4-10



d_id	c_id	date	c_id	cname	Color
22	101	2018-10-10	101	Mustang	Red
58	104	2017-5-11	104	Focus	Red
33	101	2017-1-5	101	Mustang	Red



RESULT

COUNT-d_id
3

α (Alpha) Operator: Two Arguments

$\alpha_{Arg1, Arg2}(R)$ **Arg1 represent a group(s)** to which we will apply an aggregate function Arg2 in R

Q16: Find the number of cars for each color:

$\alpha_{color, COUNT(c_id)}(CARS)$

Instance of CARS

c_id	cname	Color
101	Mustang	Red
102	Mustang	Blue
103	Malibu	Green
104	Focus	Red
105	Impala	Green



c_id	cname	Color
102	Mustang	Blue

c_id	cname	Color
101	Mustang	Red
104	Focus	Red

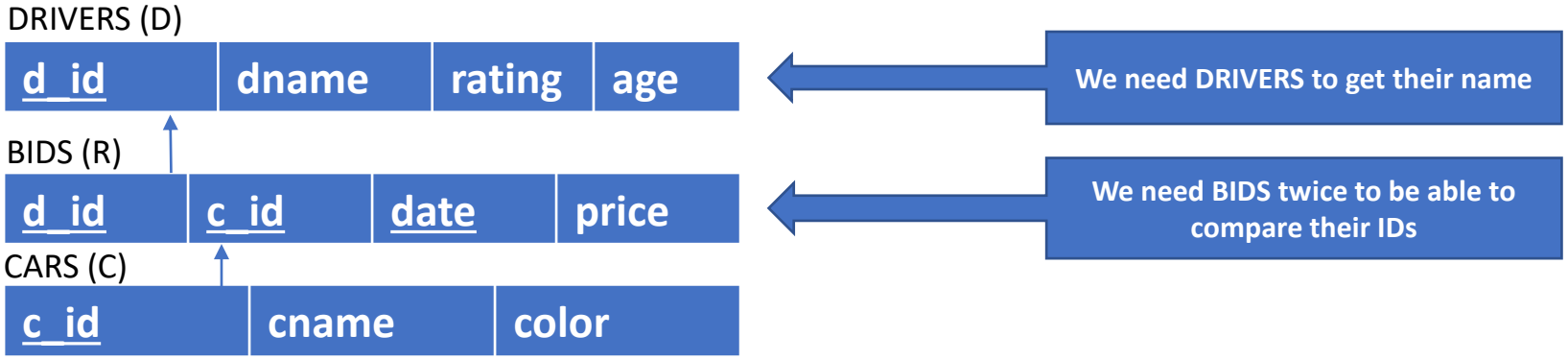
c_id	cname	Color
103	Malibu	Green
105	Impala	Green



color	count-c_id
Blue	1
Red	2
Green	2

α (Alpha) Operator: Two Arguments Example

Q9B (Revisited): Find the names of drivers who have bid on at least 2 cars



$$\rho\ (\text{TEMP}\ (2\text{-> bid1, 4->bid2}),\ \text{BIDS} \bowtie_{d_id} \text{BIDS})$$

$$\pi_{\text{dname}}((\sigma_{\text{bid1}\neq\text{bid2}}(\text{TEMP})) \bowtie \text{DRIVERS})$$

Or we could use Aggregation (Alpha) to express this in RA

α (Alpha) Operator: Two Arguments Example

Q9B (Revisited): Find the names of drivers who have bid on at least 2 cars

$$\rho\ (\ \text{TEMP},\ \alpha_{\text{d_id, COUNT(c_id)}}(\text{BIDS})\ )$$

Counts all cars IDs grouped by driver

d_id	COUNT-c_id
55	3
…	..

$$\pi_{\text{dname}}((\sigma_{\text{COUNT-c_id} \geq 2}(\textbf{TEMP}))\ \bowtie \text{DRIVERS})$$